HW1:

What is the output of the following code?

```cpp
#include <iostream>

class Base {
    virtual void method() { std::cout << "from Base" << std::endl; }
public:
    virtual ~Base() { method(); }
    void baseMethod() { method(); }
};

class A : public Base {
    void method() { std::cout << "from A" << std::endl; }
public:
    ~A() { method(); }
};

int main(void) {
    Base* base = new A();
    base->baseMethod();
    delete base;
    return 0;
}
```

from A
from A

from Base

HW2:

What is the output of the program!! And please explain what is going on?

```cpp
#include <iostream>

class Base
{
    public:
        virtual void foo() const { std::cout << "A's foo!" << std::endl; }
};

class Derived : public Base
{
    public:
        void foo() { std::cout << "B's foo!" << std::endl; }
};

int main()
{
    Base* o1 = new Base();
    Base* o2 = new Derived();
    Derived* o3 = new Derived();

    o1->foo();
    o2->foo();
    o3->foo();
}
```

A’s foo!

A’s foo!

B’s foo!

在Base 裡面的 foo() 沒有宣告 const 所以沒有成功override base 的foo ，導致即使有使用 virtual 依然是不同的function 所以會輸出 A’s foo!

HW3:

請以多型完成下列程式碼，演奏弦樂四重奏。

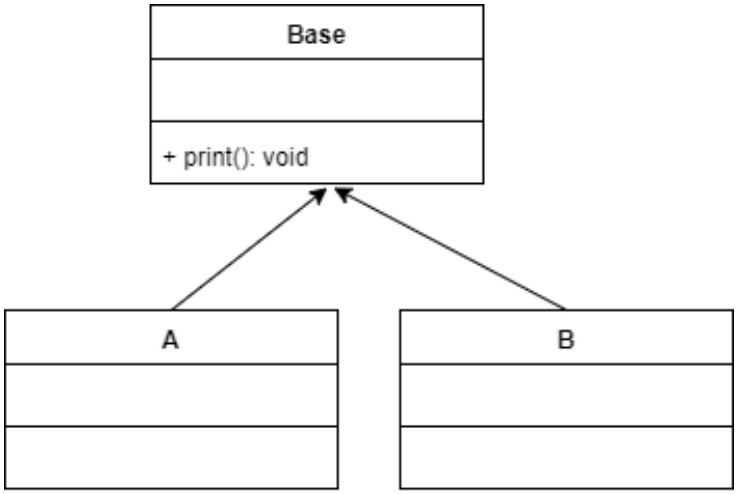
```
public class V {
	public static void main(String[] args) {
		// 弦樂四重奏
		Instrument[] stringQuartet = { new Violin(), new Violin(), new Viola(), new
Cello() };
		// play the music for me
		// implement HERE !!!
		for(int i=0;i<4;i++){
		stringQuartet[i].play;
		}

	}
}

abstract class Instrument {
	abstract public void play();
}

// 小提琴
class Violin extends Instrument {
	@Override
	public void play() {
		System.out.println("旋律");
	}
}

// 中提琴
class Viola extends Instrument {
	@Override
	public void play() {
		System.out.println("合旋");

	}
}

// 大提琴
class Cello extends Instrument {
	@Override
	public void play() {
		System.out.println("低音");

	}
}
```

HW4:

Please implement (use polymorphism) the following code to make result generate the same output.

```
#include <iostream>
#include <vector>

using namespace std;

class Base {
public:
   virtual void print() { cout << "Base" << endl; }
   virtual ~Base() {}
};

class A: public Base{
   void print() override {cout << "A" << endl;}
};

class B: public Base{
   void print() override {cout << "B" << endl;}
};


int main(void) {
   vector<Base*> bases = { new Base(), new A(), new B() };
   for (Base* b : bases) {
      b->print();
   }

   for (Base* b : bases) {
      delete b;
      b = nullptr;
   }
   bases.clear();
}
```

Please make inherit fit following class diagram.



Output:

```
$ ./a.exe
Base
A
B
```

HW5:

Modify the attachment by using polymorphism.

```
#include <stdio.h>
#include <assert.h>

#define SIZE 3

typedef struct Book Book;

typedef struct Book {
    void (*print_type) (Book *self);
} Book;

void print_Comic(Book *self){
    printf("Comic\n");
}

void print_Novel(Book *self){
    printf("Novel\n");
}

void print_Magazine(Book *self){
    printf("Magazine\n");
}

void print_book_type(Book* book) {
    assert(book != NULL);
    book->print_type(book);
}
```

```
int main(void) {
    Book books[SIZE];
    books[0].print_type = print_Comic;
    books[1].print_type = print_Novel;
    books[2].print_type = print_Magazine;

    for (int i = 0; i < SIZE; i++) {
        print_book_type(&books[i]);
    }

    return 0;
}
```

HW6:

Please write a main function (with polymorphism way) to make result generate the same output.

```
#include <iostream>
using namespace std;

class Animal {
public:
   virtual void speak() = 0;
   virtual ~Animal() {}
};
class Cat : public Animal {
public:
   void speak() {
      cout << "meow" << endl;
   }
};
class Cow : public Animal {
public:
   void speak() {
      cout << "moo" << endl;
   }
};

int main(){
   Animal *pet1 = new Cat;
   Animal *pet2 = new Cow;
   pet1->speak();
   pet2->speak();
   delete pet1;
   delete pet2;
   return 0;
}
```

Int main()
Output:

```
$ ./animal.exe
meow
moo
```